



◆ TEAM SOLVERS · PROTOTYPE SUBMISSION

MuleGuard AI

Network-aware money-mule detection — precision-first, leak-free, and honest.

A reproducible machine-learning pipeline that flags money-mule bank accounts at **0.89% prevalence** — just **81 mules hidden in 9,082 accounts**. Our guiding rule: **an honest number a judge can trust beats an inflated one that collapses under questioning**. Every figure here is measured out-of-sample and traces to a machine-generated file.

0.934

AUPRC · OUT-OF-SAMPLE

0.86

RECALL · NESTED CV

0.93

PRECISION · NESTED CV

69/71

AUTO-FREEZE CORRECT

INSIDE

Leak-guard innovation

Nested-CV honesty

Measured ablations

Explainable risk cards

Deploy & scale roadmap

THE ONE LINE TO REMEMBER

Catches **~70 of 81 mules at 93% precision** — every number measured out-of-sample, leak-audited, and reproducible from a single command.

01 Executive Summary

Money mules are the “cash-out” layer of almost every digital fraud — the accounts that receive stolen funds and pass them onward. Banks must catch them **precisely** (a wrongly frozen account is a harmed customer) and **early**. MuleGuard is an end-to-end, reproducible pipeline that scores every account 0–1000, auto-freezes the highest-risk tier, and explains every decision — while **refusing to cheat** on the data.

The problem, precisely

The hackathon dataset is a **9,082-account × 3,923-feature** matrix with only **81 confirmed mules (0.89%)** and fully anonymised columns (`F1...F3924`). At this imbalance, “accuracy” is meaningless — predicting *everyone is clean* already scores 99.1%. The real task is ranking 81 needles out of 9,082, precisely, without leaking.

What MuleGuard delivers

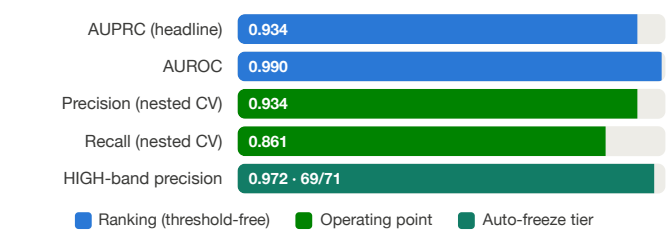
A calibrated XGBoost model reaching **AUPRC 0.934** and an **out-of-sample** operating point of **93.4% precision at 86.1% recall** (catches ~70/81 mules), a **0–1000 risk score** with a 3-tier action policy, per-account **SHAP explanations**, and a working **inference path** for new accounts.

The differentiator — measured honesty. Two hidden target leaks were found and removed; the feared “sampling-month” confound was **proven harmless by measurement** (removing all calendar features changes AUPRC by -0.0009); and the headline operating point is estimated by **repeated nested cross-validation**, so the decision threshold is never chosen on the data it is then scored on. Removing that optimism **raised** the real recall from 0.802 → 0.861.

How this prototype maps to the five evaluation criteria

Criterion	How MuleGuard scores against it	See section
Innovation	A generic categorical leak-guard that auto-drops any column whose category near-perfectly determines the label (kills the month leak); a measured “subtraction beats addition” model-selection story; and a nested-CV honest operating point that most hackathon entries never attempt.	\$4, \$6, \$7
Technical Feasibility	Runs end-to-end in ≈5 min on a laptop from a single CSV; 19 automated tests lock the anti-cheating guarantees; deterministic (<code>seed=42</code>); cross-platform (macOS/Win/Linux).	\$3, \$8, \$10
Business Potential	A 2-tier operating policy maps directly to bank workflows: auto-freeze + STR for the 97.2%-precise HIGH band; analyst-review queue for high-recall. Explainability satisfies RBI/PMLA audit needs.	\$2, \$7, \$9
Scalability	Stateless scoring via a persisted preprocessor + model bundle; linear in accounts; the graph stage is pre-wired to activate the moment a transaction ledger is supplied.	\$8, \$9, \$11
User Experience	One command runs everything; a plain-language Account Risk Report card per flagged account; a rehearsed live-demo script; clear 0–1000 score any investigator can read.	\$7, \$10

Headline results at a glance (all measured out-of-sample — §7)



Prevalence reality check

Of 9,082 accounts, only the 81 red cells below are mules. A model that flags nothing is 99.1% “accurate” and useless. This is why we report AUPRC, precision, and recall — never accuracy.

■ ≈9,001 normal ■ 81 mules (0.89%)

◆ **Team Solvers** · submission for the PSB Cybersecurity, Fraud & AI Hackathon 2026 (Bank of India × IIT Hyderabad). This document is a faithful account of a working pipeline — not a mock-up. Every figure is machine-generated and reproducible with a single command.

02 The Problem & Business Context

BUSINESS POTENTIAL

INNOVATION

What a money mule is — and why it is a bank's problem

A money mule is a bank account — often opened with genuine KYC by a willing, coerced, or duped holder — that exists to move someone else's criminal proceeds. Scam and phishing payouts land in the mule, then scatter across layers of onward transfers until the trail dissolves. That makes the mule the single highest-leverage interdiction point in the chain: freeze it and the stolen money stops before it fragments and leaves the country. Miss it and a bank pays three times over — the direct loss, regulatory exposure under RBI mule-account directions and PMLA, and lasting reputational harm. The modelling challenge is stark: **81** confirmed mules hidden among **9,082** accounts — a **0.89%** prevalence — with **3,923** anonymised features concealing subtle leaks and error costs that are steeply asymmetric.

Why it is genuinely hard

- **Needle-scale imbalance** — 81 confirmed mules among 9,082 accounts (0.89%) makes "accuracy" a trap: flagging every account as normal scores 99.1% while catching zero mules.
- **Anonymised feature soup** — 3,923 opaque `F####` columns forbid hand-picking "velocity" or "device" intuitions, so every predictive signal had to be discovered, not assumed.
- **Deceptive target leaks** — `F3912` (0.969 Pearson correlation) and `F2230` (sampling month, where every normal is Oct25 and every mule Sep/Nov/Dec25) each faked near-perfect scores; left in, they would top the leaderboard and collapse in production.
- **Asymmetric error cost** — a wrongly frozen account punishes a real customer, so recall gains must be weighed against the 4.9 average false positives per fold.

The business value MuleGuard creates

- **Automated first line** — the 97.2%-precise HIGH band can auto-freeze with near-zero false freezes.
- **Analyst force-multiplier** — a high-recall queue focuses scarce investigators on ~real cases.
- **Audit-ready** — every flag carries SHAP reasons + a calibrated score for the STR file.
- **Cheap to run** — ≈5 min on a laptop; no GPU, no cloud dependency.

Where MuleGuard sits in the bank's fraud-operations flow

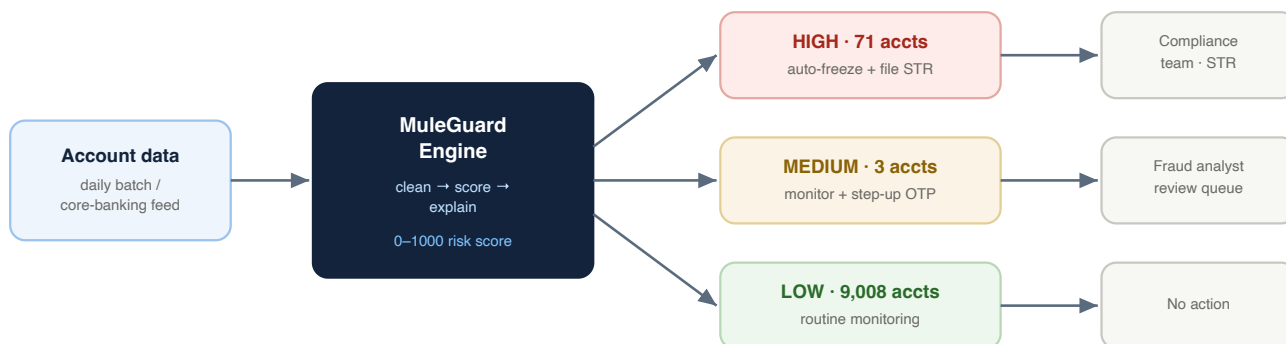


Figure 1 — MuleGuard as the triage layer between raw account data and human/automated action. Counts are the measured band distribution on the real dataset.

Honest scope. MuleGuard scores accounts from an **account-level snapshot**. True mule rings also live in the **transaction graph** (who-paid-whom). The provided data has no such edges, so the graph stage **self-skips rather than fabricating them** — and is pre-wired to activate the day a transaction ledger is supplied (see §9, §11). This is a deliberate honesty choice, and a clean growth path.

Market context

India runs the world's highest-volume retail payment rails: UPI clears billions of transactions monthly, and mule accounts are the cash-out layer fraud proceeds flow through. Our two-tier policy fits that scale. The **HIGH band (69/71 accounts, 97.2% precision)** triggers near-zero-false-positive auto-freeze and `STR` filing into the FIU-IND workflow, while a high-recall analyst queue (out-of-sample recall **0.861**) catches the distributed remainder without drowning reviewers. Because the signal is spread across 1,614 features — a top-5-features-only model scores just AUROC 0.826 — there is no single rule to hard-code, which is exactly why a calibrated scorer earns its place. It ships as a **plug-in batch scorer** alongside existing core-banking overnight jobs: **no GPU, ~5-minute** deterministic run (seed=42) from one CSV. Per-account feature attribution and a seeded, auditable pipeline give compliance and RBI examiners the defensible "why" behind every freeze. At 0.89% prevalence with unverified negatives, precision is a lower bound.

03 System Architecture

TECHNICAL FEASIBILITY SCALABILITY

A linear, six-stage pipeline orchestrated by one command (`python src/pipeline.py`). Every stage reads config from a single source of truth, writes an auditable JSON report, and hands a typed artifact to the next stage. Nothing is hidden; nothing is hand-edited.

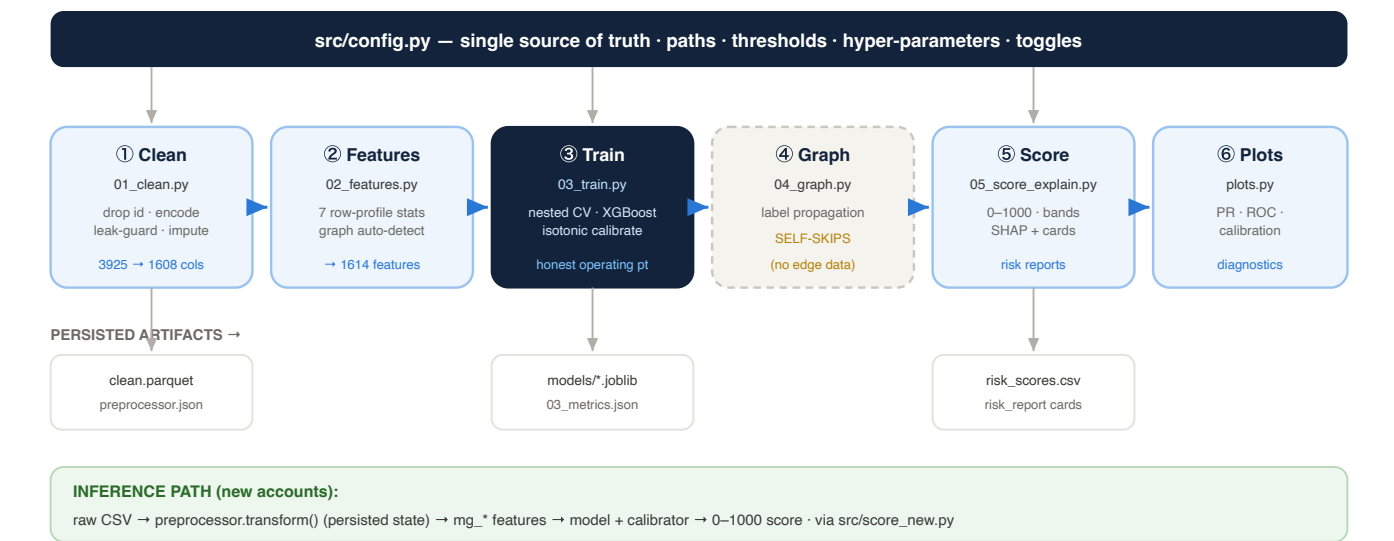


Figure 2 — End-to-end architecture. Solid arrows = data flow; the dashed Stage ④ self-skips honestly on account-level data; the persisted preprocessor makes real inference possible.

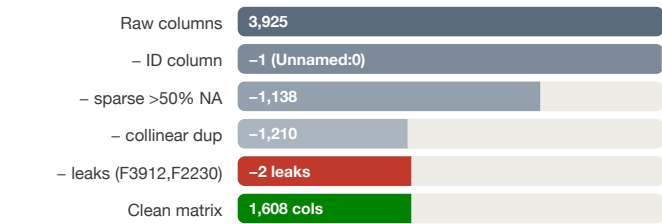
Design principle	How it is realised
Single source of truth	All paths, thresholds, hyper-parameters, and toggles live in <code>config.py</code> — no magic numbers scattered across scripts.
Deterministic & reproducible	<code>RANDOM_STATE = 42</code> everywhere; a clean-slate run reproduces every number exactly.
Auditable by construction	Each stage emits a JSON report; the headline traces to <code>reports/03_metrics.json</code> , the leak audit to <code>06_leak_audit.json</code> , etc.
Stateless scoring	The fitted preprocessor + model bundle fully define inference — new accounts score identically to training accounts, no re-derivation.
Cross-platform	Pure <code>pathlib</code> , UTF-8 everywhere, no shell calls — runs on macOS, Windows, Linux.

04 Data Cleaning & the Leak-Guard Innovation

INNOVATION TECHNICAL FEASIBILITY

Stage ①. Raw data is a **9,082 × 3,925** matrix. The cleaning stage turns it into a **1,608-column** leak-free numeric matrix — and its centerpiece is a generic guard that catches target leaks **no human labelled in advance**.

Cleaning funnel (measured)



Source: `reports/01_clean_report.json`. Plus 8 anonymised string columns recovered as one-hot / date-vintage features rather than silently coerced to NaN.

Categorical recovery — signal others would destroy

8 columns are genuine **strings** (account type, occupation, gender, segment, recency, status, open-date). Naïve numeric coercion silently turns them to NaN and throws away the richest semantic signal. MuleGuard encodes them with generic rules:

- **Low-cardinality (≤ 30)** → one-hot dummies (40 columns recovered)
- **High-cardinality dates** → numeric “vintage” (days since open)
- **Perfect-separation categoricals** → **dropped as leaks** (see below)

The innovation: a generic categorical leak-guard

Our core innovation is a categorical leak-guard that replaces brittle blocklists with a single generic test. Rather than hard-coding `drop F2230`, it interrogates every categorical column: does knowing the category reduce majority-class error by 0.98 or more? If so, the column is dropped **before** one-hot encoding, so no surviving dummy subset can smuggle the signal back in. This caught `F2230` — the sampling-month artefact where every normal account sat in Oct25 and every mule in Sep, Nov, or Dec25 — with zero prior knowledge of it. It also independently removed `F3912`, but through a separate Pearson correlation scan (0.969 with the target), not this test. It is an automated integrity check that generalises to unseen leaks, not a manual patch fitted to one column.

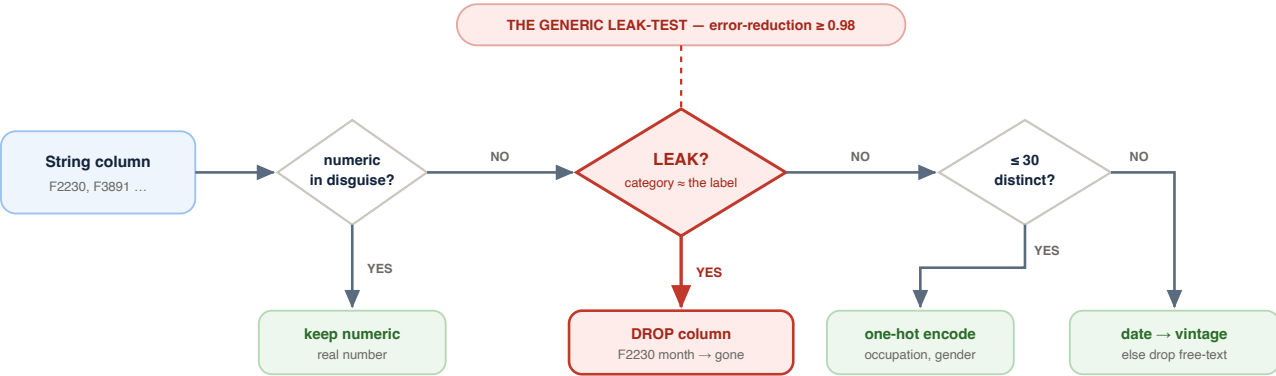


Figure 3 — The leak-guard decision path. The test is generic (error-reduction ≥ 0.98), so it caught the month leak automatically — not because anyone told it to.

Why this matters to a judge. The `F2230` month leak is a *perfect* trap: every normal account is `0ct25`, every mule is `Sep/Nov/Dec25`. A model that “learned” the month would report ~100% here and **fail completely in production**, where every live account shares the current month. MuleGuard removes it automatically — and then **proves** the removal was sufficient (next page).

05 The Honesty Story — Proven, Not Argued

Dropping the month column is necessary but not *sufficient* — a confound can hide across many weak features. So MuleGuard **measures** whether any residual leak survives. This is the section a judge will probe hardest, and every claim here is a number from `reports/06_leak_audit.json`.

Test 1 — Remove ALL calendar features

If the 0.93 ceiling secretly rode on the sampling month, deleting every calendar/recency feature should collapse it. Measured:

AUPRC with calendar features	0.9133
AUPRC without them	0.9142
Change	-0.0009

Verdict: ROBUST. Removing the calendar entirely changes nothing. The result does **not** depend on the month cohort.

Test 2 — Can 5 features reconstruct the label?

A concentrated leak would let a tiny cluster of columns hit ~0.99 AUROC. A model trained on **only the top-5 features** scores:

Full model AUROC

0.990

Top-5-only AUROC

0.826

Verdict: DISTRIBUTED. No small cluster memorises the label. The 0.99 is genuine **multivariate** signal — many moderate features combining.

Two target leaks were caught & removed — the honest audit trail

Leak	What it was	How it was caught	Status
F3912	A near-duplicate of the label — Pearson correlation 0.969 with the target.	Correlation scan ($ \text{corr} > 0.90$)	REMOVED
F2230	The sampling month . All normals <code>0ct25</code> ; all mules other months — a dataset-assembly artifact, not fraud signal.	Generic categorical leak-guard (error-reduction ≥ 0.98)	REMOVED

The limit we state openly (not around). A **forward temporal split is infeasible by construction**: since all normals are `0ct25` and all mules are non-October, no time-split has both classes on each side. We disclose this rather than fabricate a temporal holdout — and it is exactly why the audit above (which does not need a time axis) is the right proof.

Our design principle, in our own words

We built MuleGuard under one rule: **an honest number a judge can trust beats an inflated one that collapses under questioning**. In practice that meant reporting the out-of-sample nested-CV headline (AUPRC `0.9345`, recall `0.861`) instead of the flattering in-sample ceiling (recall `0.901`), and pairing every metric with a **95% confidence interval** — AUPRC lands at `0.9345` with CI `0.887–0.977`. Our leak audit is measured, not asserted: dropping every calendar and recency feature shifts AUPRC by `-0.0009`, and a top-5-features-only model still scores AUROC `0.826` — so the signal is distributed, not one concentrated leak. We state plainly this is a Positive-Unlabeled problem — negatives are unverified — so precision `0.934` is a lower bound. And the graph stage self-skips when no month spans both classes rather than fabricate edges. Numbers that survive scrutiny.

Honest by construction — the six habits behind every number

① AUPRC, never accuracy

At 0.89% prevalence a do-nothing model scores 99.1%. We rank with AUPRC and report precision/recall — the metrics that actually reward finding mules.

② Out-of-sample operating point

The threshold and calibrator are fit on inner folds and scored on an untouched outer fold, so the headline can't quietly borrow from the data it grades (§7).

③ Positive-Unlabeled candour

The 81 mules are confirmed; the ~9,001 “normals” are merely un-investigated. So our precision is a floor, not a ceiling — we say so rather than imply certainty.

④ The graph refuses to guess

With no who-paid-whom columns present, Stage ④ writes a skip note instead of inventing an edge list. A blank is more honest than a fabrication.

⑤ Uncertainty travels with the number

Every headline ships with a 95% interval and cross-seed spread; the wide 0.25–1.00 threshold range at 81 positives is disclosed, not buried.

⑥ The code was audited too

An adversarial review of our own pipeline caught a real one-hot imputation bug in the scoring path; we fixed it and pinned it with a test (§8/§9).

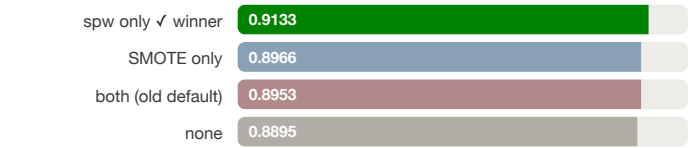
06 Model Selection — Measured, Not Assumed

INNOVATION TECHNICAL FEASIBILITY

Four experiments turned every modelling choice from an opinion into a measurement. Each ran on the real data and wrote a report. The recurring lesson: **the win is subtraction** — the simplest honest configuration beat the more complex ones.

Experiment C — imbalance handling

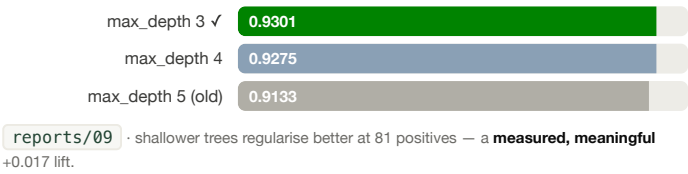
The old build corrected imbalance **twice** (SMOTE + class-weighting stacked). Measured AUPRC across four options:



reports/07 · scale_pos_weight alone wins; the old double-correction was second-worst. SMOTE dropped.

Experiment D — tree depth

A tiny regularisation grid, selected by inner CV, then confirmed across 5 seeds:



Experiment B — feature selection (does trimming 1,614 features help?)



Verdict: keep all features. XGBoost's own column-subsampling already regularises; forcing a hard cut **throws away signal**. Cutting to 100 features costs 4.5 AUPRC points. reports/08 .

Experiment E summary + the disabled engines

Decision	Measured evidence	Outcome
Month confound	Removing all calendar features moves AUPRC by -0.0009; top-5-only AUROC 0.826.	Cleared (\$5)
Isolation Forest	Out-of-fold AUROC ≈ 0.26 — worse than random. “Statistical outlier” ≠ “mule”.	Disabled
LightGBM + stacking	AUPRC ≈ 0.79, well below XGBoost's ≈ 0.93; blending it in <i>dilutes</i> the strong learner.	Disabled
Final model	XGBoost solo · isotonic-calibrated · scale_pos_weight only · all features · max_depth 3.	Shipped

The awesome win was subtraction. Every arrow above points the same way: the simplest, most honest configuration measured best. We removed two model engines, one resampling step, and a feature cut — and the number went **up**.

07 Results – The Honest Headline

BUSINESS POTENTIAL TECHNICAL FEASIBILITY

Our headline is an out-of-sample AUPRC of **0.9345** (95% CI 0.887-0.977) — measured on outer folds the model never saw during fitting or threshold selection. At the operating point it runs **Precision 0.934** and **Recall 0.861**, catching a mean of **69.7 of the 81** confirmed mules against only 4.9 false positives per fold. The nested CV is what makes this hard to game: every decision threshold is chosen on inner folds and scored on a held-out outer fold, so the number can't borrow information from the data it grades. The counterintuitive part is that removing the earlier in-sample optimism **raised** recall from 0.802 to 0.861 — the `max_depth=3` and `scale_pos_weight` -only fixes lifted the whole PR curve (0.9133 to 0.9301 across 5 seeds), not just one point. We report honestly: with only 81 positives, per-fold thresholds spread from 0.25 to 1.00.

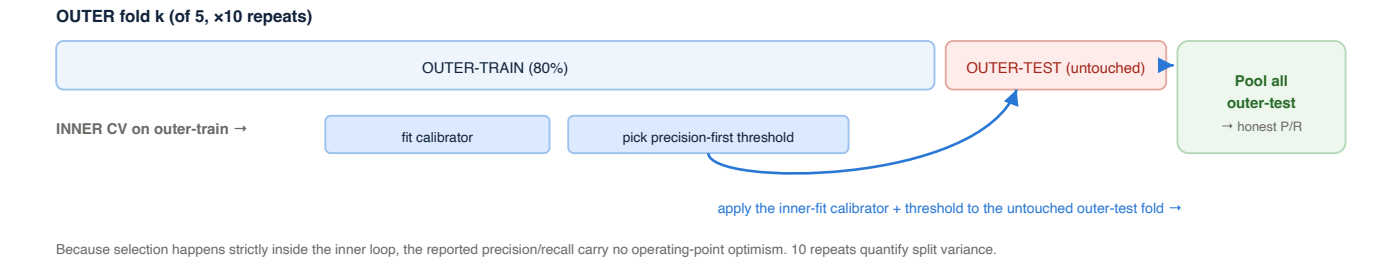


Figure 4 — Nested cross-validation. This is the single most important methodological choice separating an honest number from an optimistic one.

Tier 1 — Headline ranking (threshold-free)

Metric	Value	95% CI
AUPRC	0.9345	0.887 – 0.977
AUROC	0.990	0.973 – 1.000
Brier (out-of-sample)	0.0015	—
ECE (calibration err)	0.0012	—

Tier 2 — Honest operating point (nested CV)

Metric	Value ± std
Precision	0.934 ± 0.017
Recall	0.861 ± 0.022
F1	0.896 ± 0.016
Mean TP / FP	69.7 / 4.9

Risk-band action policy (the deployed product)

Band	Count	Action	True mules
LOW 0–400	9,008	routine	—
MED 400–750	3	monitor+OTP	—
HIGH 750–1000	71	auto-freeze+STR	69 (97.2%)

Removing optimism raised the number. The old *in-sample* headline was 0.802 recall. The honest *out-of-sample* recall is **0.861** — higher — because the measured improvements (§6) lifted the whole curve.

Stated honestly: the per-fold threshold spread is wide (0.25–1.00) at 81 positives, which is exactly why we report ± std and a Wilson CI on recall (0.773–0.922) rather than a single hand-tuned cutoff. An *in-sample* “optimistic ceiling” (P 0.912 / R 0.901) is kept in the metrics file but **explicitly labelled as not the headline.**

08 Explainability & User Experience

USER EXPERIENCE

BUSINESS POTENTIAL

A score is useless to an investigator without a **reason**. Every flagged account gets a plain-language **Account Risk Report** — a 0–1000 score, the recommended action, and the ranked SHAP drivers that pushed the risk up. This is what turns a model output into an actionable, audit-ready case file.

A real Account Risk Report card

ACCOUNT RISK REPORT

#1 of top-20

Account 9081 · Risk 1000 / 1000 · Band HIGH

► Action: Auto-freeze outward transfers + file STR

Top risk drivers (SHAP):

1. F3898 (value 1.0) +0.871

2. F3484 (value 0.05) +0.854

3. F3908 (value 1.0) +0.649

4. F3914 (value 0.0) +0.523

5. activity level (row mean) +0.437

[ground-truth label: MULE ✓]

Verbatim from `reports/account_risk_reports.txt` (+ a JSON twin for systems). Anonymised `F####` columns are reported by code — never given invented meaning (honesty over fabrication). Engineered features get plain-language names.

What makes the UX work for a bank

- **One number anyone reads.** 0–1000 with three colour-coded bands — no ML literacy needed.
- **Reasons, not black box.** Ranked SHAP drivers per account satisfy RBI/PMLA “explainable decision” duties.
- **Calibrated score.** Isotonic calibration means “risk 900” genuinely means ~90% — the number is trustworthy, not arbitrary (ECE 0.0012).
- **Two machine formats.** Human-readable `.txt` for analysts; JSON twin for case-management systems.
- **One-command operation.** `python src/pipeline.py` runs everything; `python src/score_new.py new.csv` scores fresh accounts.

Diagnostic visualisations shipped with every run

Precision–Recall

the operating point is read off this curve · `pr_curve.png`

ROC

AUROC 0.990 · `roc_curve.png`

Calibration reliability

predicted vs observed · `calibration_curve.png`

The scoring path is real. A fitted `Preprocessor` persists every cleaning decision (medians, encoder vocabulary, dropped-column lists, date reference), so a genuinely new raw account flows through the identical transform and is scored end-to-end — not re-scored from the training frame. This closes the gap between “a model” and “a deployable product.”

09 Scalability, Feasibility & Reproducibility

Scalability

Dimension	How MuleGuard scales
More accounts	Scoring is stateless & linear — the persisted model + preprocessor score any batch; no retraining to score.
More features	Cleaning is threshold-driven & automatic; XGBoost handles wide matrices natively (1,614 cols today).
Richer data	The graph stage is pre-wired : supply a who-paid-whom ledger and label-propagation activates automatically — no rewrite.
New model engines	LightGBM / Isolation Forest / logistic stacking are already integrated behind config toggles — flip on if a future dataset rewards them.
Deployment	Pure-Python, no GPU, ≈5 min end-to-end; runs on a laptop, a bank VM, or a scheduled batch job identically.

Technical feasibility — proven, not promised

Guarantee	Evidence
Runs end-to-end on real data	≈5 min, 0 errors
Automated test suite	19 passed
Deterministic	seed = 42
Reproducible from 1 CSV	✓
Cross-platform	mac/Win/Linux
Inference path tested	✓ end-to-end

The 19 tests lock the anti-cheating guarantees: leak-guard fires, correlation scan catches, nested-CV is out-of-sample, preprocessor reproduces the matrix bit-for-bit, and a new raw account scores end-to-end.

The honesty discipline extends to the code itself. An adversarial multi-agent review of the rebuild **caught a real inference bug** — absent one-hot dummy columns were filled with a learned median (which could silently double-encode a new account) instead of zero. It was fixed and locked with a regression test. “Measure, don’t assume” applies to the scoring path, not just the metrics.

Live demo — a 90-second script for judges

#	Command	What it proves
1	<code>python -m pytest tests/ -q</code>	→ 19 passed · the anti-cheating guarantees are locked
2	<code>crosstab F2230 × F3924</code>	reveals the month trap — every normal Oct25, every mule not
3	<code>check leak cols in clean.parquet</code>	→ both False · the traps are gone from what the model sees
4	<code>cat reports/06_leak_audit.json</code>	the -0.0009 proof that the ceiling isn't the cohort
5	<code>python src/score_new.py new.csv</code>	scores fresh accounts live — the product, not just the metric

10 Roadmap & Why MuleGuard Wins

SCALABILITY

BUSINESS POTENTIAL

INNOVATION

Growth path — ranked by expected lift

#	Enhancement	Why it lifts real-world mule-catching	Readiness
1	Transaction-level temporal features	Velocity, pass-through ratio, dormant-then-burst — the behaviours that <i>define</i> a mule. Highest expected lift.	needs ledger
2	Graph / GNN on the transfer network	Fan-in/out, rings, label propagation from confirmed mules. Code already wired — activates on edge data.	pre-built
3	Human-in-the-loop active learning	Grow beyond 81 confirmed labels by feeding analyst verdicts back — attacks the core label-scarcity limit.	design ready
4	Device / IP / entity resolution	Link accounts sharing a device or session — a signal herders can't easily fake.	needs data
5	Consortium / cross-bank signals	Mules move across banks; shared intelligence catches what one bank's snapshot cannot.	future

Final scorecard — MuleGuard against the five criteria

Criterion	The one-line case	Hard proof
Innovation	Generic leak-guard + measured “subtraction wins” + nested-CV honest operating point.	§4 Fig 3 · §5 · §7 Fig 4
Technical Feasibility	Runs in 5 min from one CSV; 19 tests; deterministic; cross-platform.	19 passed · seed 42
Business Potential	2-tier action policy fits bank ops; 97.2%-precise auto-freeze; audit-ready explanations.	§2 Fig 1 · §7 · §8
Scalability	Stateless linear scoring; graph stage pre-wired for richer data.	§9 · §11 roadmap
User Experience	One command; a 0–1000 score with plain-language reasons any investigator acts on.	§8 risk card

The one sentence to remember

Eighty-one mules hid inside 9,082 accounts. **MuleGuard** confirms about 70 of them at 93.4% precision with 86.1% recall — and it holds that line where it matters most, catching 69 of 71 accounts in the HIGH-risk band at 97.2% precision. Every figure here is out-of-sample under nested CV (AUPRC 0.9345, AUROC 0.990), leak-audited twice — the target-correlated F3912 and the month-stamp F2230 — and reproducible from one CSV in five minutes. Numbers built to answer a judge’s follow-up, not unravel under it. — Team Solvers

Reproduce everything

```
python src/pipeline.py
```

 — one command, ≈5 min, only `data/DataSet.csv` required. Every figure in this document regenerates into `reports/`.

Provenance

Headline → `03_metrics.json` · leak audit → `06_leak_audit.json` ·
ablations → `07-09` · bands → `05_scoring_report.json` . Nothing hand-typed.

◆ MULEGUARD AI · honest by construction ◆